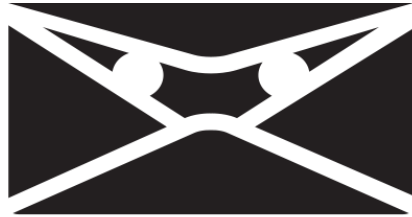


Testing



InvoiceNinja

Using POSTMAN Tool

Team Members:	○ Israa Bamarouf ID 2211682 ○ Joud Mozahem ID 2212477 ○ Hajar Habiburahman ID 2211350 ○ Sarah Bahashwan ID 2211101 ○ Shahad Algarni ID 2217309
Section:	SE 2

Instructor:

- Dr.Muna Altherwi

Contents

Introduction	2
Part 1	2
Modeling Structure and Coverage Criterion	2
Functionality 1 Apply Unapplied Amount to Invoice	3
Step 1 Identify Testable Functions	3
Step 2 Find All Parameters, Inputs and Characteristics	3
Step 3 Model the Input Domain	3
Step 4 Apply a Test Criterion to Choose Combinations of Values	4
Model 1: Input Space Partitioning	4
Test Cases Model 1	7
Test Case 1	7
Test Case 2	8
Test Case 3	9
Test Case 4	10
Test Case 5	11
Test Case 6	12
Test Case 7	13
Test Case 8	14
Test Case 9	15
Model2: Control Flow Graph	17
Coverage Criterion: Node Coverage	19
Coverage Criterion: Edge Coverage	19
Coverage Criterion: Prime Path Coverage	19
Test Cases Model 2	20
Test Case 1	20
Test Case 2	21
Test Case 3	22
Test Case 4	23
Test Case 5	24
Test Case 6	25
Part 2	29
Introduction to POSTMAN	29
Configuration and Setup	29
How it's Used	31
Testing Results	32
Summary:	41
Conclusion	42
References	42

Introduction

This project aims to explore and communicate in-depth knowledge about software testing tools. Effective software testing is crucial for ensuring the reliability and functionality of applications. This document outlines our testing approach for the invoice ninja application using the POSTMAN tool.

[Invoicing co API documentation.](#)

[Invoicing co website](#)

Part 1

Modeling Structure and Coverage Criterion

Software Name: Invoice Ninja Web/App.

Tool Name: POSTMAN.

Modeling Structure: Input Space Partitioning.

Coverage Criteria: Base Choice Coverage Criteria, Multiple Base Choice Coverage Criteria, Each choice Coverage Criteria.

Constraints:

company_gateway:

- Must exist and be a valid company gateway object.

gateway_type_id:

- Must be valid ID.

amount (gateway fee amount):

- Must be positive.
- Zero, negative, non-numeric values are invalid.

payment_hash_string:

- Must be valid not empty string.

Functionality 1 | Apply Unapplied Amount to Invoice

Step 1 Identify Testable Functions

The testable function we used is `addGatewayFee()` method. Used to apply gateway fee based on gateway type, amount, and payment hash.

Step 2 Find All Parameters, Inputs and Characteristics

Inputs:

- `company_gateway`
- `gateway_type_id`
- `amount`
- `payment_hash_string`

Parameters: Each input has parameters for valid, invalid, or empty, missing conditions.

Characteristics: `company_gateway` validity, `gateway_type_id` validity, `amount` validity, `payment_hash_string` validity.

Step 3 Model the Input Domain

Valid Inputs:

Partition 1: Valid `company_gateway`, Valid `gateway_type_id`, positive `amount`, and valid `payment_hash_string`.

Invalid Inputs:

Partition 2: Invalid `company_gateway`.

Partition 3: Invalid `gateway_type_id`.

Partition 4: Amount is zero or negative.

Partition 5: Amount is non-numeric.

Partition 6: `payment_hash_string` is empty or null.

Empty Inputs:

Partition 7: Missing `company_gateway`.

Partition 8: Missing `gateway_type_id`.

Partition 9: Missing `amount`.

Partition 10: Missing `payment_hash_string`.

Step 4 Apply a Test Criterion to Choose Combinations of Values

Characteristic	Block 1	Block 2	Block 3
q ₁ = company_gateway validity	Valid (Valid Object)	Invalid	Missing
q ₂ = gateway_type_id validity	Valid (Valid ID)	Invalid	Missing
q ₃ = amount validity	Valid (Positive)	Invalid	Missing (or non-numeric)
q ₄ = payment_hash_string validity	Valid (Valid string)	Invalid	Missing

Model 1: Input Space Partitioning

Base choice coverage Criteria:

Base Choice: (Valid, Valid, Valid, Valid)

We selected this as the base choice because it represents the happy path.

The test requirements to satisfy Base choice coverage criteria:

(Invalid, Valid, Valid, Valid)
(Valid, Invalid, Valid, Valid)
(Valid, Valid, Invalid, Valid)
(Valid, Valid, Valid, Invalid)
(Missing, Valid, Valid, Valid)
(Valid, Missing, Valid, Valid)
(Valid, Valid, Missing, Valid)
(Valid, Valid, Valid, Missing)

Multiple Base Choice Coverage Criteria:

Base Choice: (Valid, Valid, Valid, Valid)

We selected this as the base choice because it represents the happy path.

The test requirements to satisfy Multiple Base Choice Coverage criteria:

Changing q_1 = company_gateway validity

- (Invalid, Valid, Valid, Valid)
- (Missing, Valid, Valid, Valid)

Changing q_2 = gateway_type_id validity

- (Valid, Invalid, Valid, Valid)
- (Valid, Missing, Valid, Valid)

Changing q_3 = amount validity

- (Valid, Valid, Invalid, Valid)
- (Valid, Valid, Missing, Valid)

Changing q_4 = payment_hash_string validity

- (Valid, Valid, Valid, Invalid)
- (Valid, Valid, Valid, Missing)

Each Choice Coverage Criteria:

Max (number of blocks) = 3

- (Valid, Valid, Valid, Valid)
- (Invalid, Invalid, Invalid, Invalid)
- (Missing, Missing, Missing, Missing)

Test Cases Model 1

Test Case 1

Case ID	TC_01
Title	Base-Choice – All Inputs Valid
Description	Verify that addGatewayFee succeeds when all parameters are valid.
Precondition	A valid Bearer token is set and company_gateway = 1, gateway_type_id = 2, amount = 100, and payment_hash_string = “abc123” are provided.
Assumption	The specified gateway and company exist and can accept fees.
Test Data	• company_gateway: 1 • gateway_type_id: 2 • amount: 100 • payment_hash_string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 200 OK with a success message and the new fee record.

Test Case 2

Case ID	TC_02
Title	Base-Choice – Negative Amount
Description	Verify that addGatewayFee rejects a negative amount.
Precondition	A valid Bearer token is set and company_gateway = 1, gateway_type_id = 2, amount = -50, and payment_hash_string = “abc123” are provided.
Assumption	Amount must be greater than zero.
Test Data	• company_gateway: 1 • gateway_type_id: 2 • amount: -50 • payment hash string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with an error about amount needing to be greater than 0.

Test Case 3

Case ID	TC_03
Title	Base-Choice – Missing Payment Hash
Description	Verify that addGatewayFee rejects when payment_hash_string is omitted.
Precondition	A valid Bearer token is set and company_gateway = 1, gateway_type_id = 2, amount = 100 are provided but payment_hash_string is omitted.
Assumption	The payment_hash_string field is mandatory.
Test Data	• company_gateway: 1 • gateway_type_id: 2 • amount: 100
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with an error stating payment_hash_string is required.

Test Case 4

Case ID	TC_04
Title	Multiple-Base – Invalid Gateway & Type
Description	Verify that addGatewayFee reports errors for both an invalid company_gateway and an invalid gateway_type_id.
Precondition	A valid Bearer token is set and amount = 100, payment_hash_string = “abc123” are provided along with company_gateway = 9999 and gateway_type_id = 8888.
Assumption	Both fields are validated in the same request.
Test Data	• company_gateway: 9999 • gateway_type_id: 8888 • amount: 100 • payment_hash_string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with errors for invalid company_gateway and invalid gateway_type_id.

Test Case 5

Case ID	TC_05
Title	Multiple-Base – Zero Amount & Missing Company
Description	Verify that addGatewayFee reports errors for amount = 0 and missing company_gateway.
Precondition	A valid Bearer token is set and gateway_type_id = 2, amount = 0, payment_hash_string = “abc123” are provided but company_gateway is omitted.
Assumption	Both company_gateway and a positive amount are required.
Test Data	• gateway_type_id: 2 • amount: 0 • payment hash string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with errors for missing company_gateway and amount needing to be greater than 0.

Test Case 6

Case ID	TC_06
Title	Multiple-Base – Non-numeric Amount & Missing Hash
Description	Verify that addGatewayFee reports errors for a non-numeric amount and missing payment_hash_string.
Precondition	A valid Bearer token is set and company_gateway = 1, gateway_type_id = 2 are provided but amount = “one_hundred” and payment_hash_string is omitted.
Assumption	Amount must be numeric and payment_hash_string is mandatory.
Test Data	• company_gateway: 1 • gateway_type_id: 2 • amount: one_hundred
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with errors for non-numeric amount and missing payment_hash_string.

Test Case 7

Case ID	TC_07
Title	Each-Choice – Missing Company Gateway
Description	Verify that addGatewayFee rejects when company_gateway is omitted.
Precondition	A valid Bearer token is set and gateway_type_id = 2, amount = 50, and payment_hash_string = “abc123” are provided but company_gateway is omitted.
Assumption	The company_gateway field is required.
Test Data	• gateway_type_id: 2 • amount: 50 • payment hash string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with an error stating company_gateway is required.

Test Case 8

Case ID	TC_08
Title	Each-Choice – Missing Gateway Type
Description	Verify that addGatewayFee rejects when gateway_type_id is omitted.
Precondition	A valid Bearer token is set and company_gateway = 1, amount = 50, and payment_hash_string = “abc123” are provided but gateway_type_id is omitted.
Assumption	The gateway_type_id field is required.
Test Data	• company_gateway: 1 • amount: 50 • payment hash string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data.
Expected Result	The API returns HTTP 422 Unprocessable Entity with an error stating gateway_type_id is required.

Test Case 9

Case ID	TC_09
Title	Each-Choice – Missing Amount
Description	Verify that addGatewayFee rejects when amount is omitted.
Precondition	A valid Bearer token is set and company_gateway = 1, gateway_type_id = 2, payment_hash_string = “abc123” are provided but amount is omitted.
Assumption	The amount field is required.
Test Data	• company_gateway: 1 • gateway_type_id: 2 • payment hash string: abc123
Test Steps	Send a POST request to /api/v1/gateway_fees with the Test Data in JSON format.
Expected Result	The API returns HTTP 422 Unprocessable Entity with an error stating amount is required.

Functionality 2 | Apply Unapplied Amount to Invoice

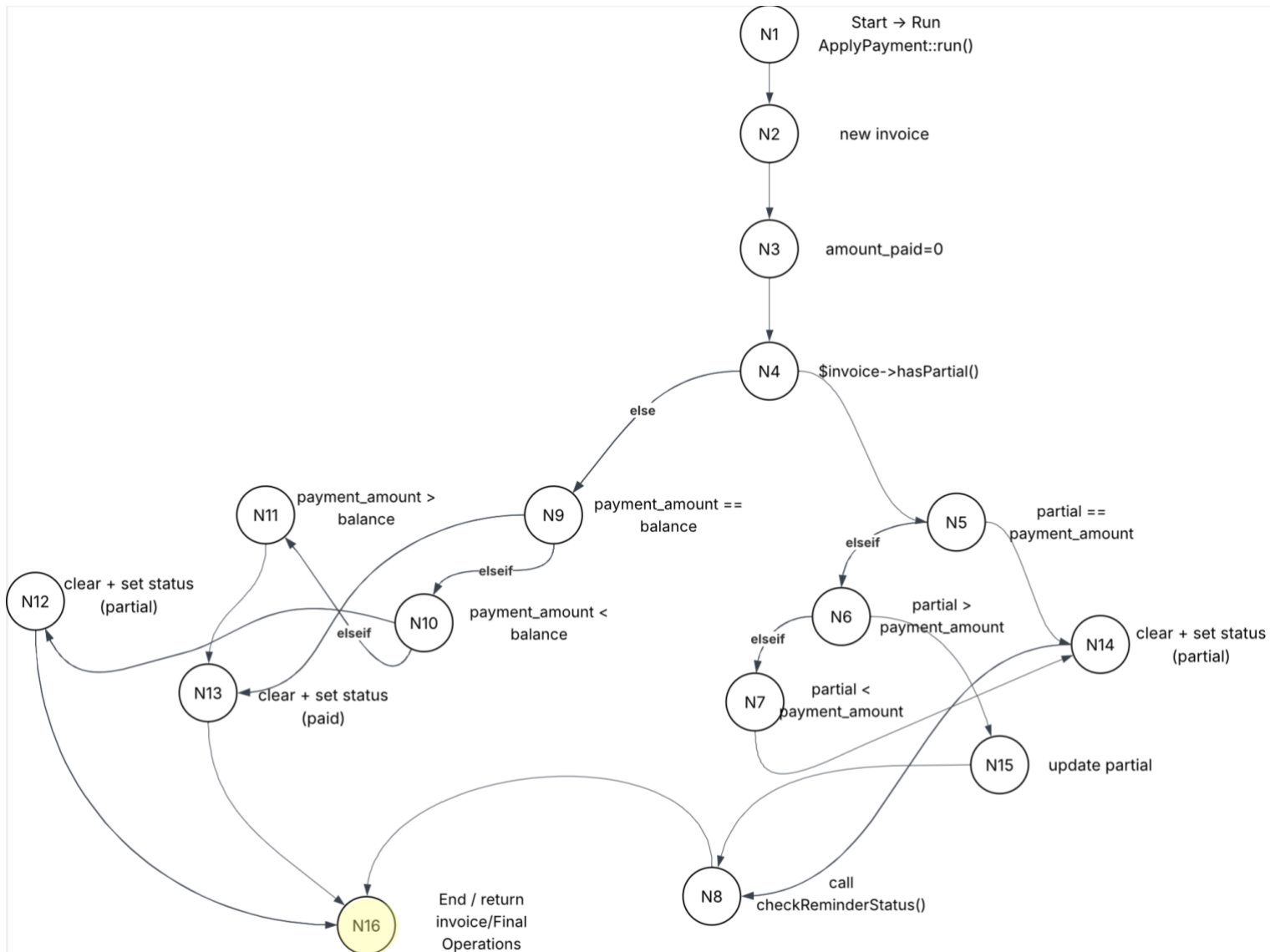
Identify Testable Functions

The testable function in this case is run() method. Used to Apply Full, Partial, and Unapplied Payments to Invoice

Modeling Structure: Control Flow Graph (CFG).

Coverage Criteria: (Node Coverage, Edge Coverage, Prim Path Coverage).

Model2: Control Flow Graph



Node Number	CFG Node	Short ID	Description
[1]	N1	Start	Entry point of ApplyPayment::run() method
[2]	N2	Refresh Invoice	Calls \$invoice->fresh('client') to reload invoice data
[3]	N3	Init amount paid	Initializes \$amount paid = 0
[4]	N4	hasPartial() decision	First branching: checks if invoice has a partial amount
[5]	N5	Partial == Amount	Compares if partial amount matches payment
[6]	N6	Partial > Amount	Checks if partial is greater than payment
[7]	N7	Partial < Amount	Checks if partial is less than payment
[8]	N8	Reminder Status	Calls checkReminderStatus() and saves the invoice
[9]	N9	payment_amount == balance	Checks if payment_amount is exactly equal to the invoice balance
[10]	N10	payment_amount < balance	Checks if payment_amount is less than the invoice balance
[11]	N11	payment_amount > balance	Checks if payment_amount is greater than the invoice balance
[12]	N12	clearPartial + setStatus Partial (from balance)	Triggered when payment_amount < invoice->balance and there's no partial; clears any previous partial, sets status to Partial, and updates the balance and paid-to-date.
[13]	N13	clearPartial + setStatus Paid (from balance)	Triggered when payment_amount == or > invoice->balance; clears partial if present, sets status to Paid, and updates the balance and paid-to-date.
[14]	N14	clearPartial + setStatus Partial (from partial match)	Triggered when hasPartial() and partial == payment_amount; clears partial, sets due date, sets status to Partial, updates balance and paid-to-date.
[15]	N15	updatePartial (payment less than partial)	Triggered when hasPartial() and partial > payment_amount; updates the partial amount, updates balance and paid-to-date without clearing partial.
[16]	N16	Final Operations	Final operations: update payment ledger, client balance, apply workflow, and return invoice

Coverage Criterion: Node Coverage

$TR = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16\}$

Test Paths:

- **T1:** [1,2,3,4,9,10,12,16]
- **T2:** [1,2,3,4,9,10,11,13,16]
- **T3:** [1,2,3,4,5,6,7,14,8,16]
- **T4:** [1,2,3,4,5,6,15,8,16]

Coverage Criterion: Edge Coverage

$TR = (1,2), (2,3), (3,4), (4,5),$
 $(5,14), (14,8), (8,16), (5,6),$
 $(6,15), (15,8), (6,7), (7,14),$
 $(4,9), (9,13), (13,16), (9,10),$
 $(10,12), (12,16), (10,11), (11,13)$

Test Paths:

- **T1:** [1,2,3,4,5,14,8,16]
- **T2:** [1,2,3,4,5,6,15,8,16]
- **T3:** [1,2,3,4,5,6,7,14,8,16]
- **T4:** [1,2,3,4,9,13,16]
- **T5:** [1,2,3,4,9,10,12,16]
- **T6:** [1,2,3,4,9,10,11,13,16]

Coverage Criterion: Prime Path Coverage

TR and Test Paths:

- [1, 2, 3, 4, 5, 6, 7, 14, 8, 16]
- [1, 2, 3, 4, 5, 6, 15, 8, 16]
- [1, 2, 3, 4, 9, 10, 11, 13, 16]
- [1, 2, 3, 4, 9, 10, 12, 16]
- [1, 2, 3, 4, 5, 14, 8, 16]
- [1, 2, 3, 4, 9, 13, 16]

Test Cases Model 2

Test Case 1: Edge coverage T1: [1,2,3,4,5,14,8,16]

Case ID	TC_01
Title	Apply payment exactly equal to partial amount
Description	When the invoice has a partial and the payment amount is exactly equal to the partial amount and Ensures system allows partial payment and reflects remaining balance.
Precondition	• Invoice has a partial amount. • Payment amount is equal to the partial amount. • Client exists, Invoice with balance of 1000 exists
Assumption	• The invoice exists with a partial balance. • Client and invoice are valid.
Test Data:	• amount: 600 • invoices: [{"invoice_id": "Le3QBZ3Qb6 ", "amount": 600}]
Test Steps:	Send POST request to /api/v1/payments with the partial amount
Expected Result	Payment applied partially, invoice remains open, client balance = 400

Test Case 2: Edge coverage T2: [1,2,3,4,5,6,15,8,16]

Case ID	TC_02
Title	Apply Payment Less than Partial Amount
Description	When the invoice has a partial and the payment amount is less than the partial amount.
Precondition	• The invoice has a partial. • Payment amount is less than the partial amount.
Assumption	• The invoice exists with a partial balance.
Test Data:	• payment_amount: 500 • invoice.partial: 1000
Test Steps:	Send a POST request with the payment amount of 500.
Expected Result	• Update the partial. • Check reminder state

Test Case 3: Edge coverage T3: [1,2,3,4,5,6,7,14,8,16]

Case ID	TC_03
Title	Apply Payment Greater than Partial Amount
Description	When the invoice has a partial and the payment amount is greater than the partial amount.
Precondition	• Invoice has a partial balance. • Payment amount is greater than the partial amount.
Assumption	• The invoice exists with a partial balance.
Test Data:	• payment_amount: 1500 • invoice.partial: 1000
Test Steps:	Send a POST request with the payment amount of 1500.
Expected Result	• Partial is cleared • Invoice status set to partial. • Check reminder state.

Test Case 4: Edge coverage T4: [1,2,3,4,9,13,16]

Case ID	TC_04
Title	Apply Full Payment to Invoice
Description	Verify payment is fully applied to one open invoice
Precondition	- The invoice has no partial balance. - Payment amount equals the full invoice balance. - client exists, and one open invoice with balance 1000 exists for the client.
Assumption	The invoice exists and has a balance due.
Test Data:	- client_id: DdwjNNGmb1 - amount: 200 - invoices: [{"invoice_id": "wdL7MKWwbj ", "amount": 200}]
Test Steps:	Send POST request to /api/v1/payments with above data
Expected Result	Payment is applied fully to invoice, marked paid, client balance = 0

Test Case 5: Edge coverage T5:[1,2,3,4,9,10,12,16]

Case ID	TC_05
Title	Apply Payment Less Than Full Invoice
Description	When the invoice has no partial and payment amount is less than the Invoice balance.
Precondition	• The invoice has no partial balance. • Payment amount is less than the Invoice balance. • Client exists • Invoice balance = 1000
Assumption	Partial payment is accepted - System sets status to Partial
Test Data:	"invoice_id": " vbm01dgEey", "amount": 600}]
Test Steps:	Send POST request to /api/v1/payments with the above data
Expected Result	• Paymwnt applied partialy invoice remains open, client balance =400

Test Case 6: Edge coverage T6:[1,2,3,4,9,10,11,13,16]

Case ID	TC_06
Title	Apply overpayment to single invoice
Description	Validates how system handles amount greater than invoice balance.
Precondition	• Invoice has no partial • Invoice balance = 300 • Client exists
Assumption	• System accepts overpayment and stores excess as unapplied.
Test Data:	• invoices: [{"invoice_id": "abc125", "amount": 400}]
Test Steps:	Send a POST request with the payment amount of 400.
Expected Result	• 300 applied to invoice • 100 stored as unapplied

Test Case 7:

Case ID	TC_07
Title	Apply payment with missing required fields (no client ID)
Description	Test validation when required fields are omitted
Precondition	None.
Assumption	Client ID is required.
Test Data:	amount: 1000 (no client_id)
Test Steps:	Send POST request to /api/v1/payments with the above data (no client ID)
Expected Result	error validation message: "client ID is required".

Test Case 8:

Case ID	TC_08
Title	Apply payment with invalid client ID
Description	Verify that the API rejects non-existent client ID.
Precondition	No client exists with ID “invalid123”
Assumption	Client ID is mandatory.
Test Data:	- client ID: "invalid123" - amount: 1000
Test Steps:	Send POST request to /api/v1/payments with above data (non-existent ID)
Expected Result	Error message “client not found” or validation error.

Test Case 9:

Case ID	TC_09
Title	Apply payment with no invoice
Description	Validates storing a payment without linking it to any invoice.
Precondition	Client exists
Assumption	Client has unpaid invoices (not linked in this test)
Test Data:	- amount: 100 - invoices: [] (empty)
Test Steps:	Send POST request to /api/v1/payments with above data (without invoice details).
Expected Result	Payment stored as unapplied, visible in client balance

Test Case 10:

Case ID	TC_10
Title	Apply zero-amount payment
Description	System rejecting non-accurate invoice amounts
Precondition	Valid client
Assumption	Minimum amount should be > 0
Test Data:	- client_id: "DdwjNNGmb1 " - amount: 0
Test Steps:	Send POST request to /api/v1/payments with the above data (amount = 0)
Expected Result	Error validation message: "amount must be greater than 0".

Test Case 11:

Case ID	TC_11
Title	Send payment with invalid amount type
Description	Confirms amount field must be numeric.
Precondition	A valid client exists.
Assumption	Non-numeric values should trigger validation error.
Test Data:	- amount: "abc" - client_id: "DdwjNNGmb1 "
Test Steps:	Send POST request to /api/v1/payments with the above data
Expected Result	error message: "amount must be numeric"

Test Case 12:

Case ID	TC_12
Title	Apply Payment with Negative Amount
Description	Tests system validation when user attempts to send a negative payment.
Precondition	A valid client Invoice exists and is unpa.
Assumption	The API will respond with an error for invalid payment amounts.
Test Data:	• payment_amount: -100 • invoice.balance: 1000
Test Steps:	Send a POST request to /api/v1/payments with negative amount
• Expected Result	• System returns error • Payment is rejected • No changes made to the invoice

Part 2

Introduction to POSTMAN

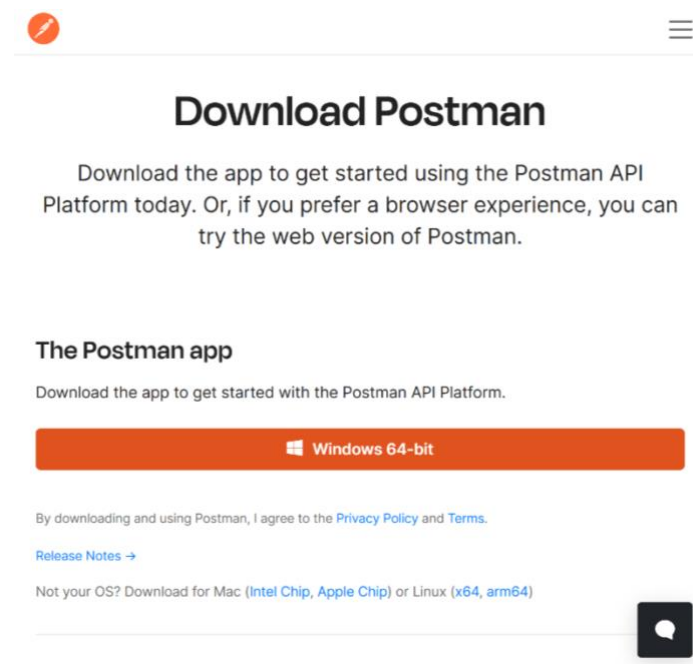
Postman is an open-source platform designed for API development, testing, and monitoring. It allows developers and testers to send requests (like GET, POST, PUT, DELETE), inspect responses, create test cases, and organize them into collections. POSTMAN is used in many different contexts, such as:

- **Manual Testing:** The developers and testers can manually verify API endpoints to ensure they work as expected.
- **Automated Testing:** Writing tests for APIs to automatically check functionality and performance.
- **API Development:** Creating and testing APIs during the development phase to ensure they meet requirements.
- **Collaboration:** Teams can share their API collections, facilitating communication and understanding of the API's functionality.[1]

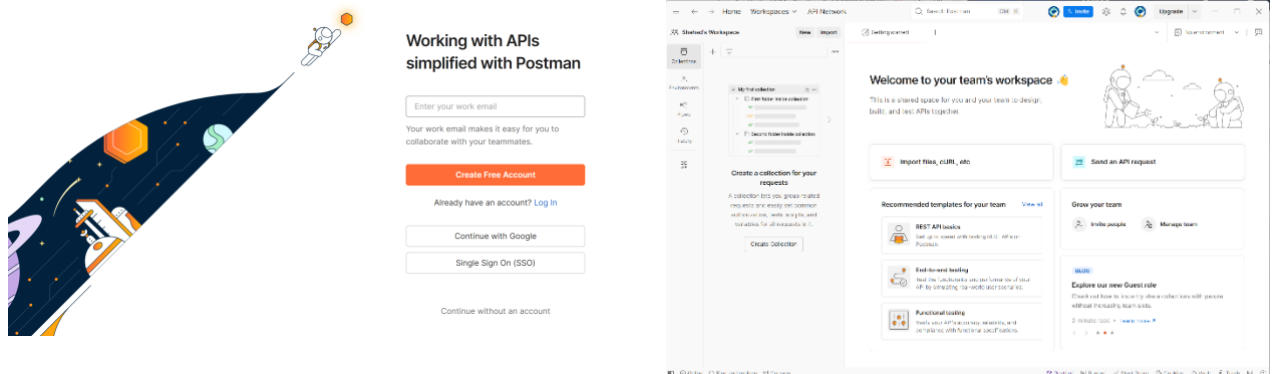
Configuration and Setup

1-Download and Install POSTMAN :

- Download it from <https://www.postman.com/downloads/> and install on your computer.

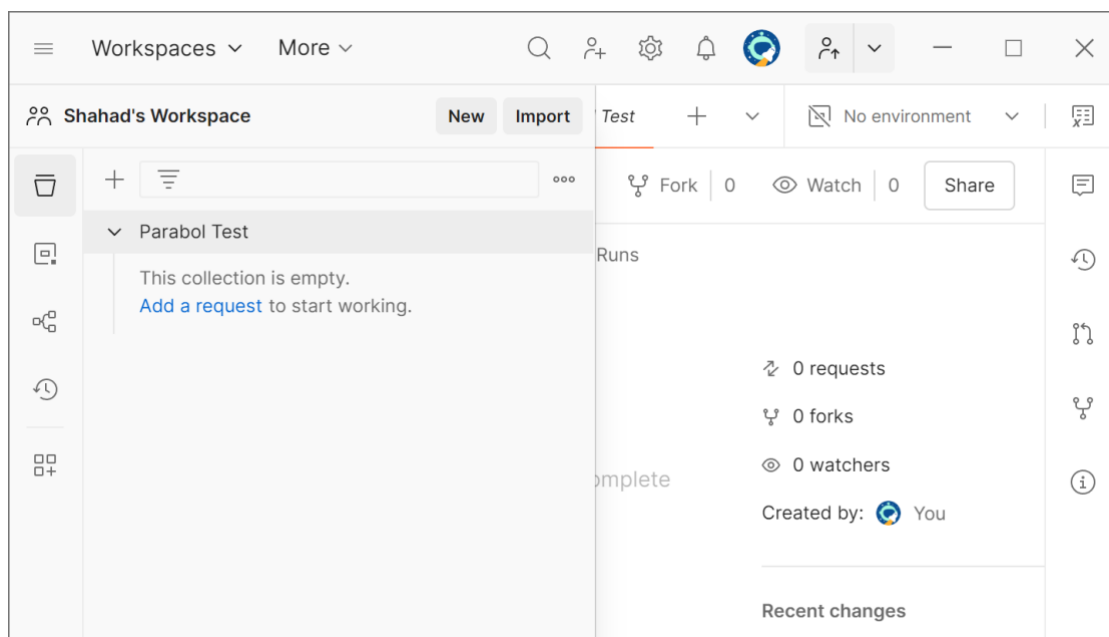


- Create Account and log in to start using POSTMAN:



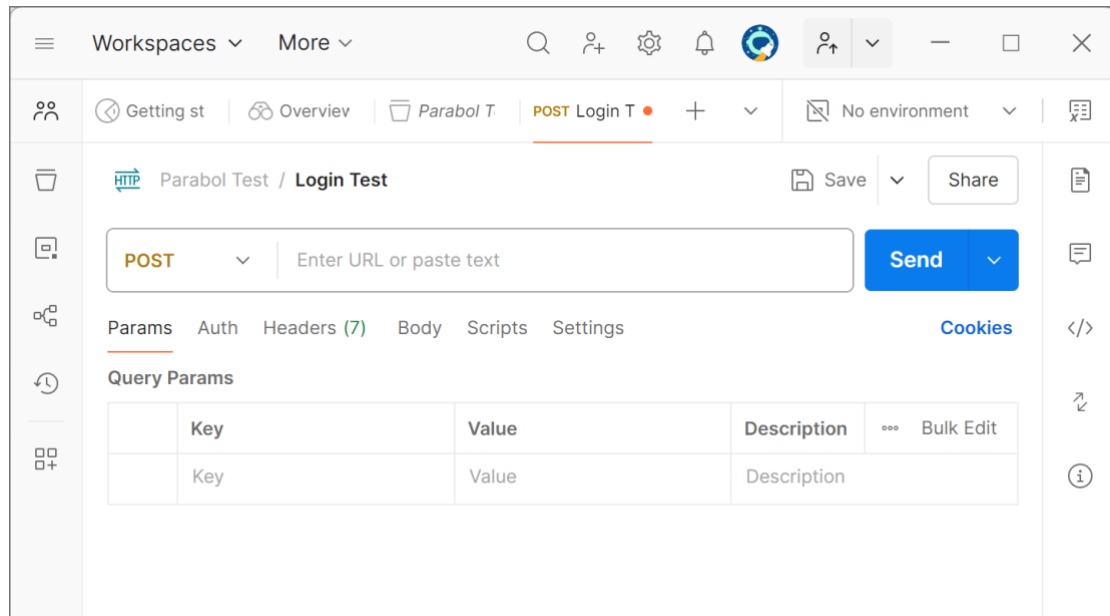
2- Create a New Collection:

- Click on “Collections” in the left sidebar.
- Click “New Collection” and provide a name.



3- Create a New Request:

- Within collection, click “Add Request”.
- Name the request and select the request type.



How it's Used

Postman is used to send API requests and validate responses as part of software testing.

1. Select the request method (GET, POST) and enter the API endpoint.
2. Configure headers and body.
3. Send the request to interact with the API.
4. Review the response status, data, and performance metrics.
5. Add test scripts to automatically validate responses.
6. Use the Collection Runner to execute multiple test cases efficiently.
7. Manage variables like base URLs.

Testing Results

On Model 2 [graph]

Test 1:

Case ID#: TC_04

Expected result:

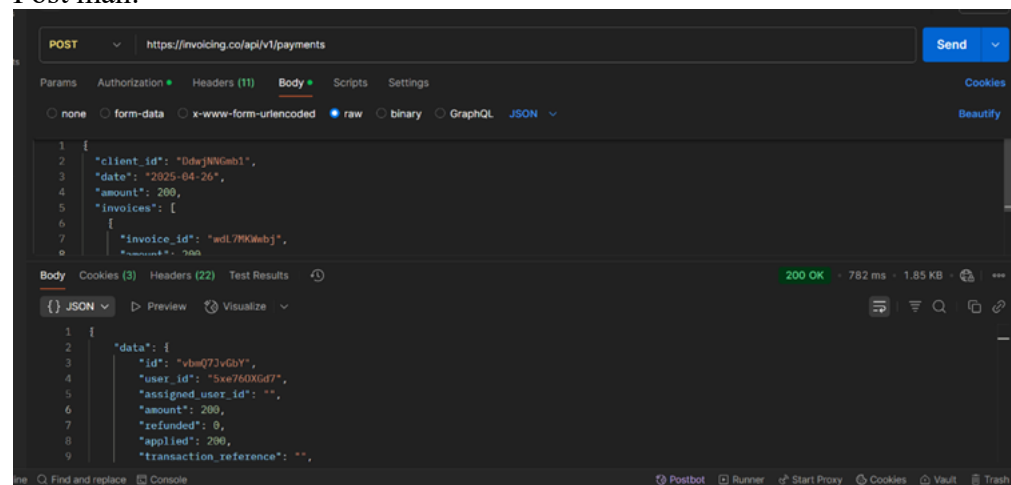
Payment is applied fully to invoice, invoice marked paid, client balance = 0

Actual result:

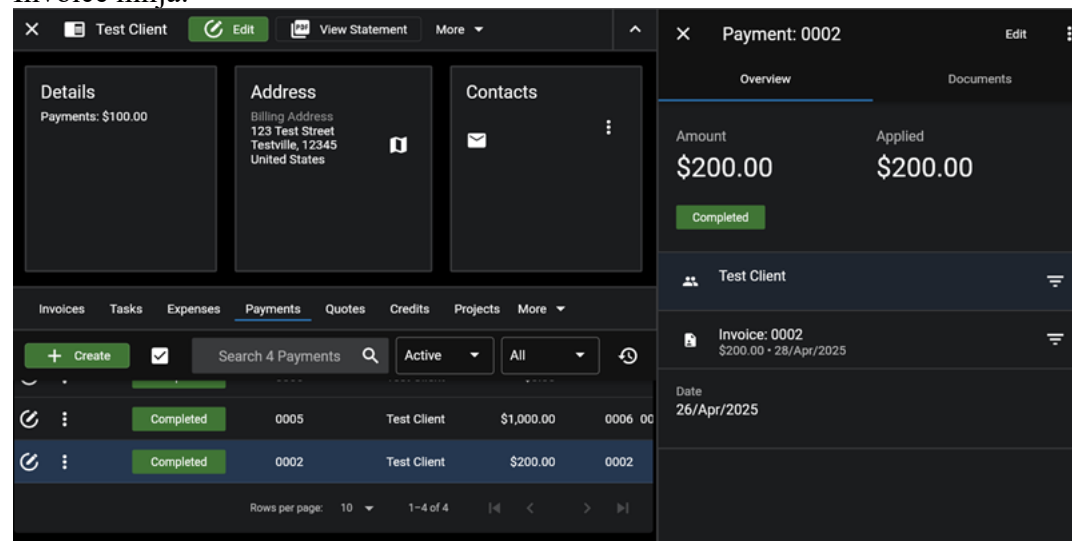
Payment is applied fully to invoice, invoice marked paid, client balance = 0, and a confirmation email was sent to the owner.

Test Result: pass

Post man:



Invoice ninja:



TC2:

Case ID#: TC_05

Expected result:

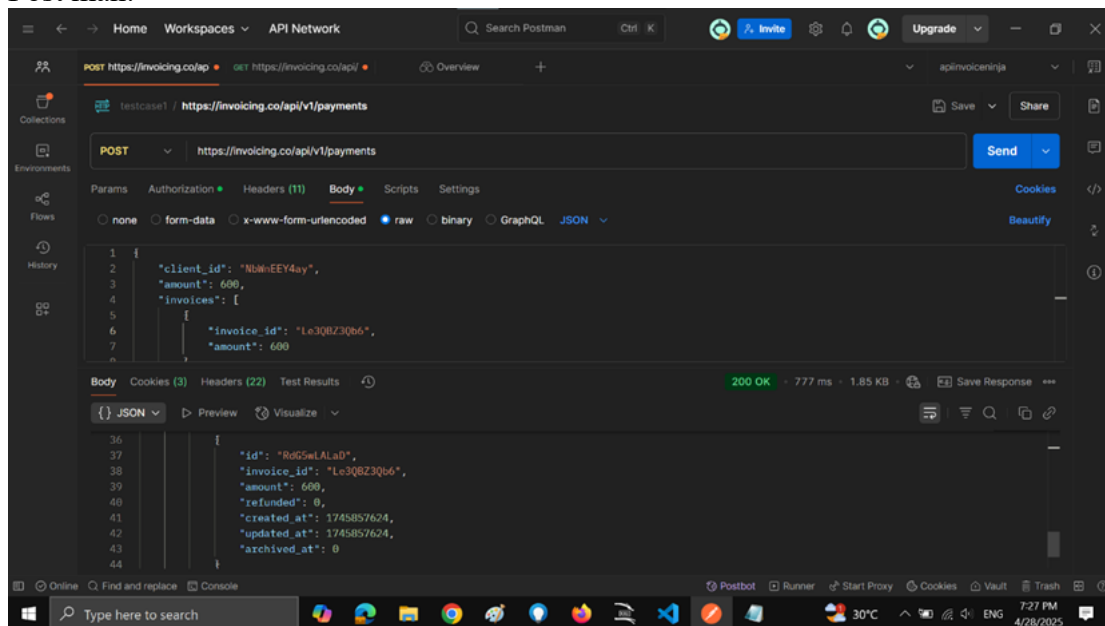
Payment applied partially, invoice remains open, client balance = 400

Actual result:

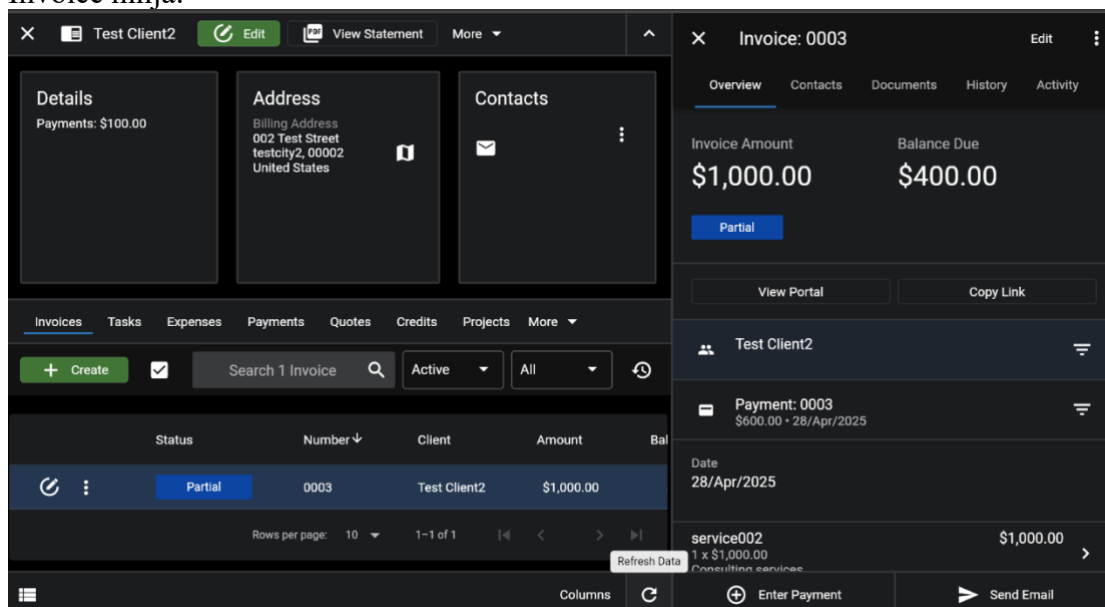
Payment applied partially, invoice remains open, client balance = 400

Test Result: pass

Post man:



Invoice ninja:



TC3:

Case ID#: TC_09

Expected result:

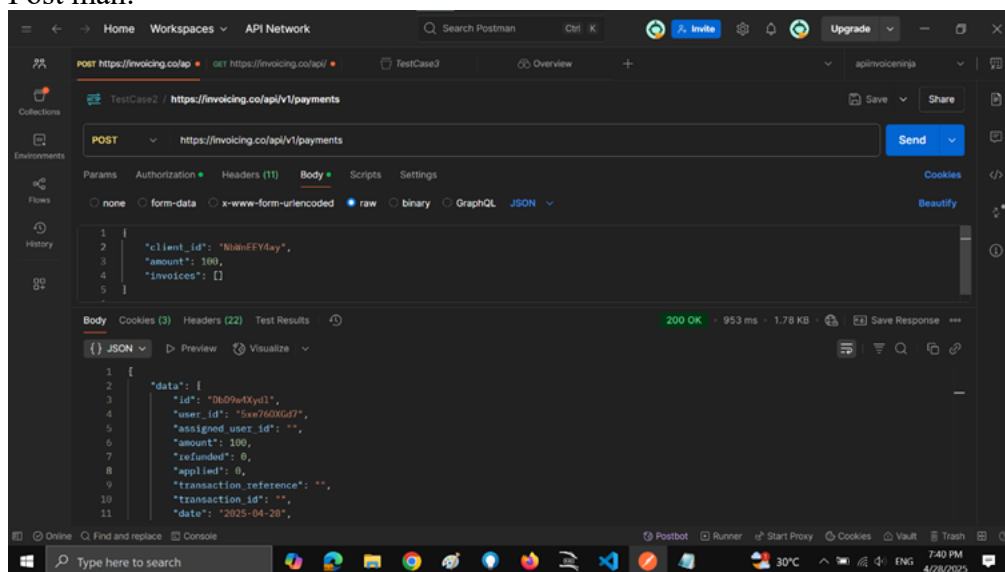
Payment stored as unapplied, visible in client balance.

Actual result:

Payment is added to the client's paid amount, but the remaining balance doesn't change

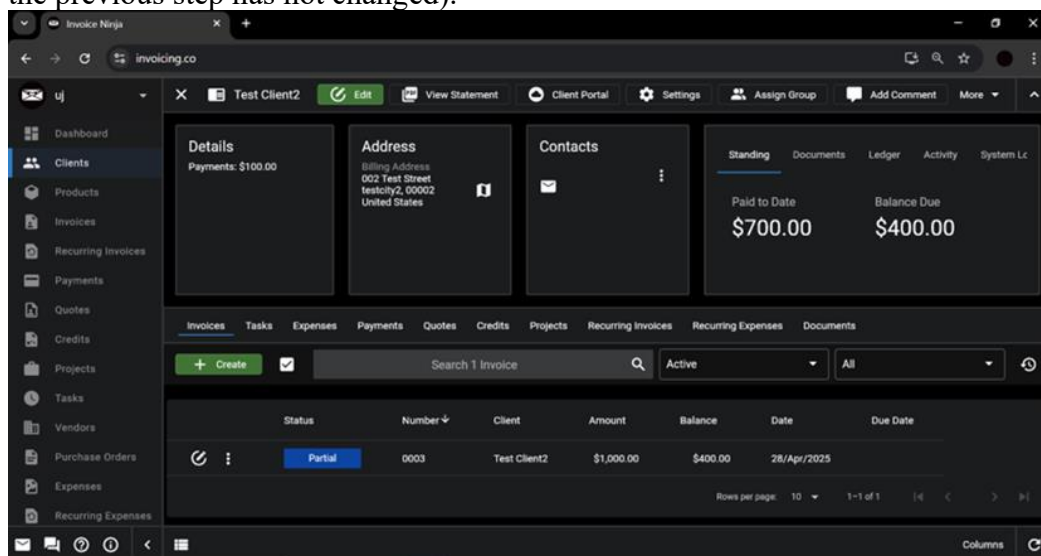
Test Result: fail

Post man:



Invoice ninja:

(notice how the 'Paid to Date' became 700 instead of 600, but the balanced due from the previous step has not changed).



TC4:

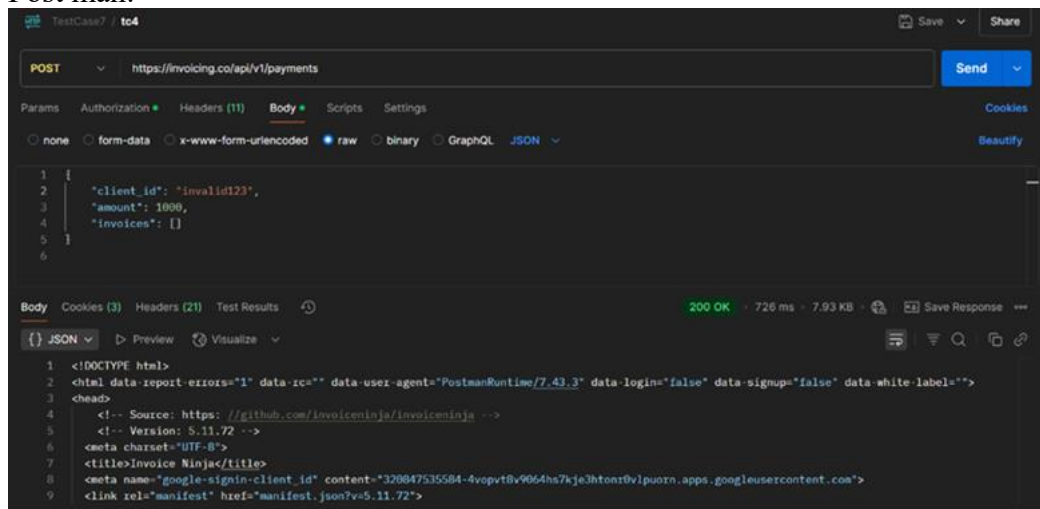
Case ID#: TC_08

Expected result: error message “client not found” or validation error.

Actual result:

Server error

Post man:



Invoice ninja: (-)

Test Result: pass

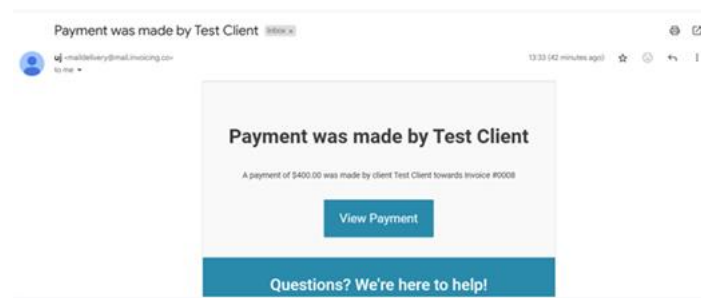
TC5:

Case ID#: TC_06

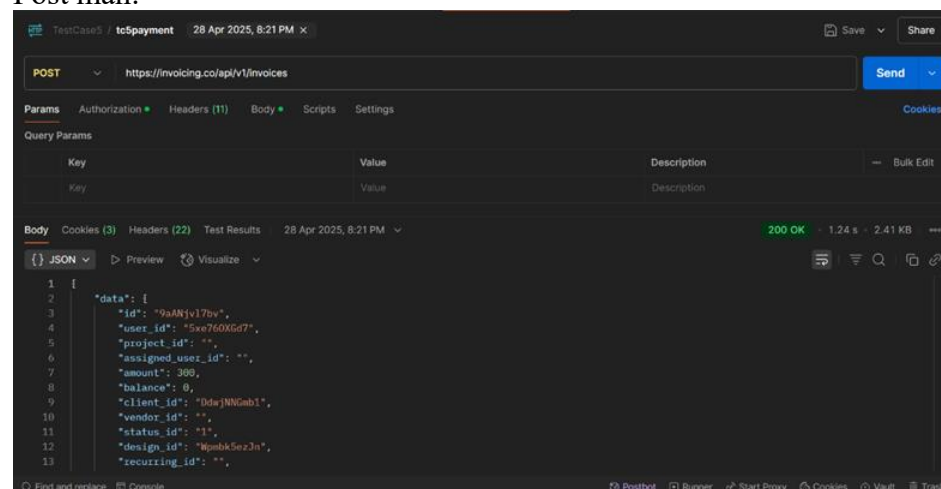
Expected result: 300 is applied, 100 stored as unapplied, invoice closed

Actual result: Payment is processed, and the owner of the company was sent a confirmation email, but no overpaid balance is found, and the status is listed as partially unapplied.

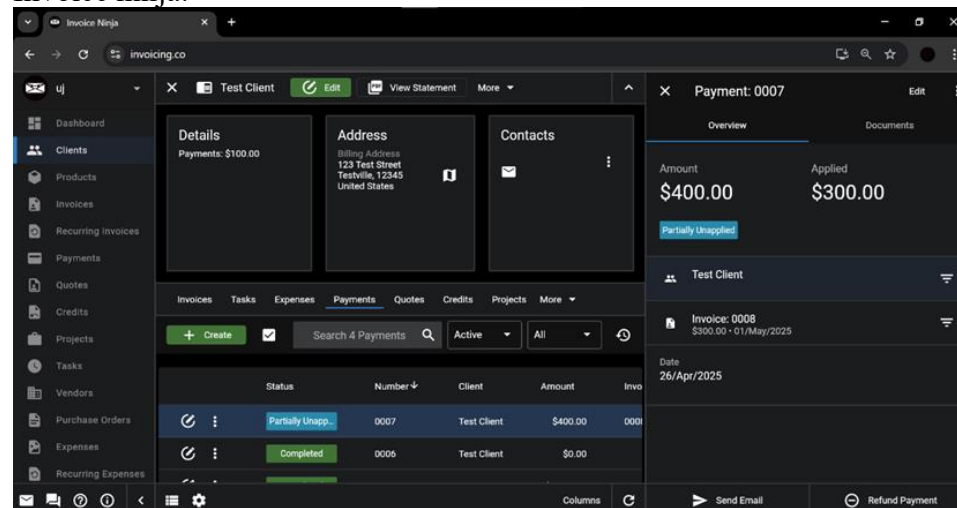
Test results: fail



Post man:



Invoice ninja:



TC6:

Case ID#: TC_12

Expected result:

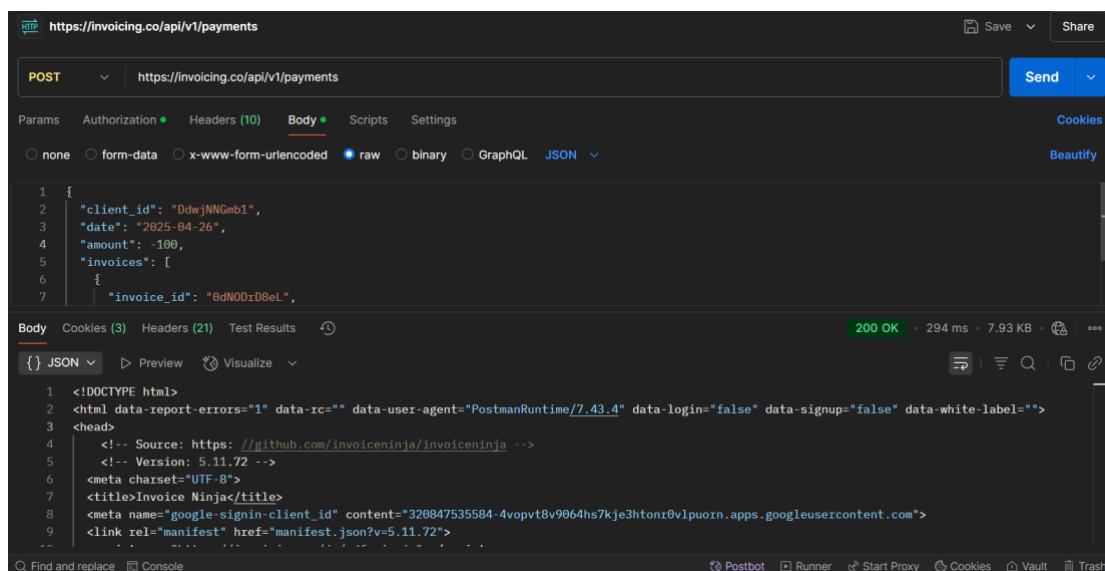
Both invoices marked paid, full payment distributed accordingly.

Actual result:

Error

Test Result: pass

Post man:



Invoice ninja: (-)

TC7:

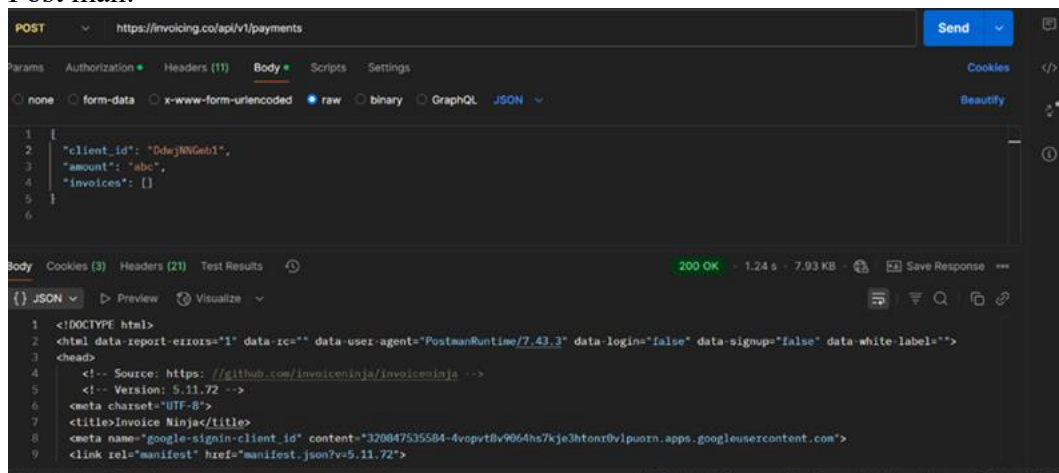
Case ID#: TC_11

Expected result: error message: “amount must be numeric”

Actual result: Error

Test Result: pass

Post man:



Invoice ninja: (-)

TC8:

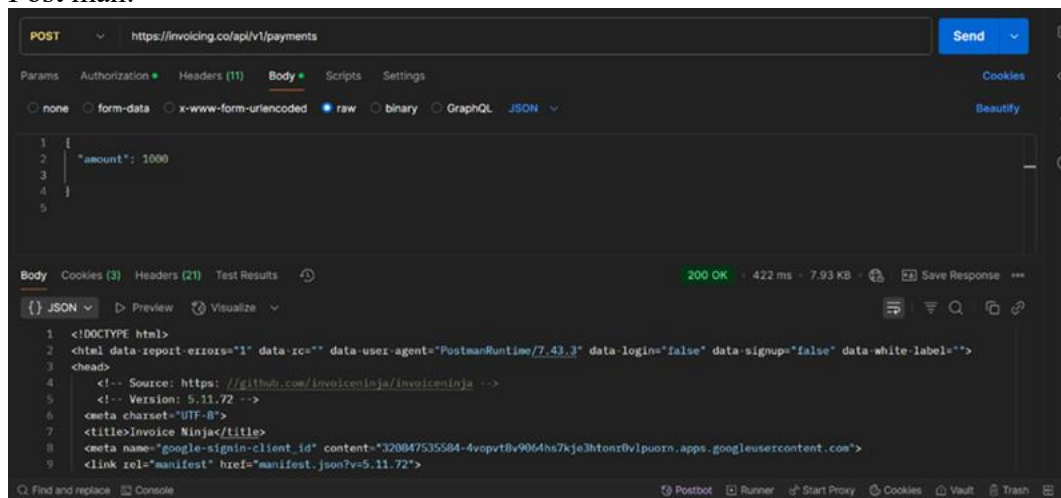
Case ID#: TC_07

Expected result: error validation message: "client_id is required".

Actual result: Error

Test Result: pass

Post man:



Invoice ninja: (-)

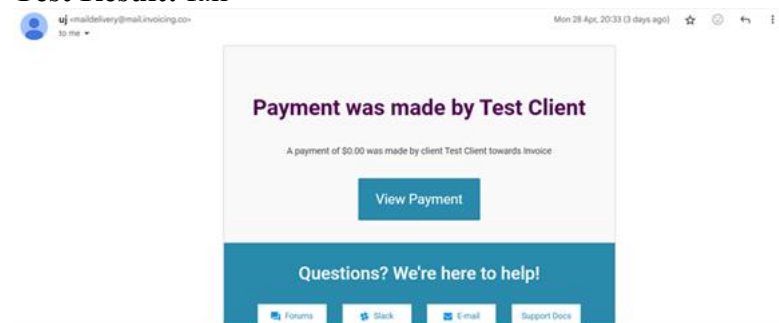
TC9:

Case ID#: TC_10

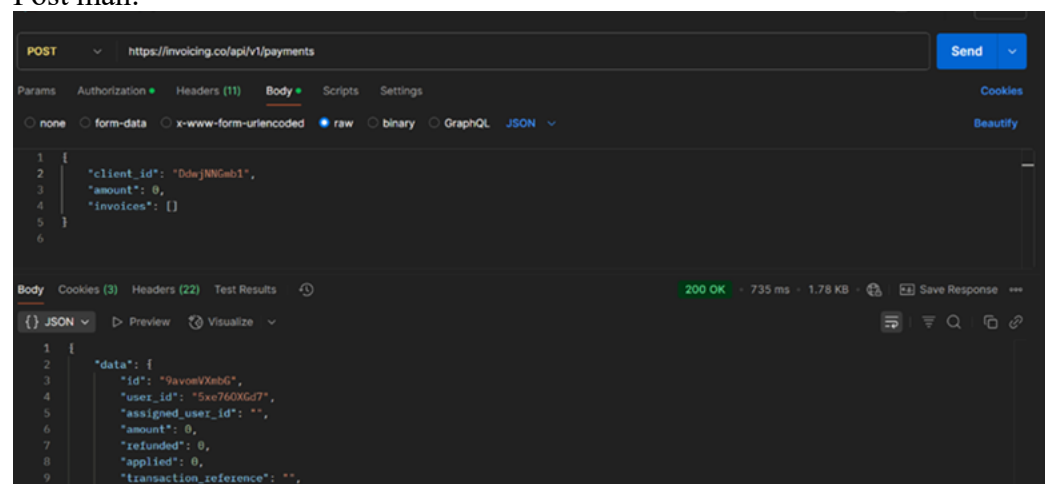
Expected result: error validation message: “amount must be greater than 0”.

Actual result: Payment was successful, the amount of 0.0\$ was added to the dashboard and a confirmation message was sent to the owner.

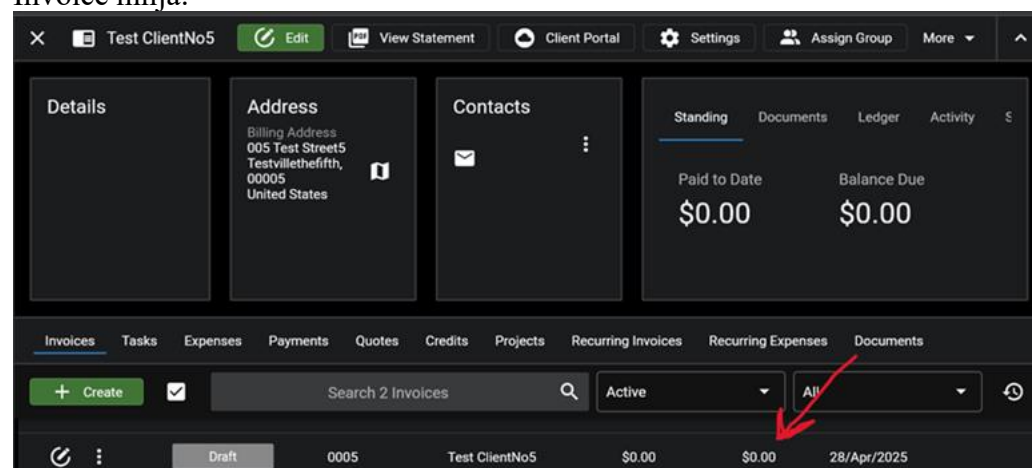
Test Result: fail



Post man:



Invoice ninja:



Summary:

Test Case #	Description	Expected Result	Test Result
TC1	Verify payment is fully applied to one open invoice	Payment is applied fully to invoice, marked paid, client balance = 0	Passed
TC2	When the invoice has no partial and payment amount is less than the invoice balance	Payment applied partially, invoice remains open, client balance = 400	Passed
TC3	Validates storing a payment without linking it to any invoice.	Payment stored as unapplied, visible in client balance	Failed
TC4	Verify API rejects non-existent client ID.	Error message “client not found” or validation error.	Passed
TC5	Validates how system handles amount greater than invoice balance.	300 is applied, 100 stored as unapplied, invoice closed	Failed
TC6	Tests system validation when user attempts to send a negative payment	Error message	Passed
TC7	Confirms amount field must be numeric.	Error	Passed
TC8	Test validation when required fields are omitted	Error, validation message	Passed
TC9	System should reject payments with amount = 0	Error, validation message: “amount must be greater than 0”.	Failed

Conclusion

Postman makes API testing easier and faster.

Using Postman, we have tested the Invoice Ninja APIs and found that they worked correctly for different payment cases, postman helped us check that the system handles both successful actions and input errors properly. Some results were unexpected as the website did not work as we expected in a few cases. This shows how important the testing process is to make sure everything works correctly.

References

- [1] Postman, "Postman Learning Center," [Online]. Available: <https://learning.postman.com>. [Accessed: Apr 11, 2025].